INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI

Department of Computer Science and Engineering

M.Tech Project Report

(MTP Phase II)

Anonymity Preserving Decoupled Distributed Authentication for Multihop IoT Networks

---

Submitted by:

Anup Kulkarni
Roll No: [244101005]
M.Tech (CSE)

Supervisor:

Dr. Manas Khatua
Associate Professor
Dept. of CSE, IIT Guwahati

Academic Year 2024–2026

CERTIFICATE

*This is to certify that the work contained in this thesis entitled "**Anonymity Preserving Decoupled Distributed Authentication for Multihop IoT Networks**" is a bonafide work of **Anup Kulkarni** (**Roll No. 244101005**), carried out in the Department of Computer Science and Engineering, Indian Institute of Technology Guwahati under my supervision and that it has not been submitted elsewhere for a degree.*

Supervisor: **Dr. Manas Khatua**

Associate Professor,

Jun, 2026

Department of Computer Science & Engineering,

Guwahati.

Indian Institute of Technology Guwahati, Assam.

Acknowledgements

I would like to take this opportunity to express my heartfelt gratitude to my thesis supervisor, Dr. Manas Khatua, from the Department of Computer Science and Engineering at IIT Guwahati, for his invaluable guidance, constant support, and encouragement throughout this project. His insightful advice and continuous efforts have not only guided me in this work but also imparted valuable lessons that will assist me in my future career.

I would also like to thank Sreeparna Das, Saurav Baidya, Amritesh Singh, and Abhishek Tiwari for their support and collaboration throughout this work. I am also deeply grateful to my parents and friends for their unwavering support and cooperation. Their wisdom and encouragement have been a constant source of strength in my life. Finally, I would like to reiterate my sincere thanks to Dr. Manas Khatua, without whose guidance this project would have remained a distant dream.

Abstract

The primary objective of this work is to identify, address, and validate critical security and privacy gaps in DAuth, a lightweight decoupled distributed authentication scheme for multihop IoT networks. Two significant vulnerabilities were identified. First, the real device identity is transmitted in plaintext over the public channel during every authentication session. This exposure enables any passive adversary to track devices across sessions, correlate repeated authentication attempts, and infer network behaviour, which directly violates modern privacy requirements. Second, when a key-exchange packet is lost in transit, the device and authentication server hold divergent session states, causing all subsequent authentication attempts to fail and forcing the device through a costly full re-enrolment.

To address both vulnerabilities, this work proposes an extension of DAuth that preserves device anonymity and tolerates desynchronisation. Per-session pseudonym rotation replaces the real device identity in all open-channel communications with a rotating pseudonym that is refreshed after each successful key exchange, so that successive pseudonyms are computationally unlinkable. A dual-state mechanism maintained at the authentication server retains both the current and the immediately preceding session state, enabling seamless recovery from a single packet-loss event without re-enrolment. Formal verification under a standard adversary model confirms authentication correspondence, session key secrecy, forward secrecy, anonymity preservation, and offline guessing resistance. COOJA/Contiki-NG simulations on a 100-node RPL network show a 42.8 % reduction in per-device energy over LAAKA and 32.4 % over Zhou et al., with consistent gains across all network sizes and authentication server pool configurations. Hardware evaluation on Raspberry Pi 4B confirms 52.5 % and 14.2 % energy reductions over LAAKA and Zhou et al., respectively; DAuth achieves the lowest hardware overhead as it carries no pseudonym operations, while the proposed scheme remains well below both LAAKA and Zhou et al.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

The Internet of Things (IoT) has expanded rapidly and now underpins critical applications across smart manufacturing, power grids, remote health monitoring, and intelligent transport systems. These deployments rely on large numbers of resource-constrained devices communicating over shared wireless channels that are inherently exposed to eavesdropping. Securing such networks against impersonation, replay, man-in-the-middle, key compromise, and device-capture attacks is therefore a fundamental operational requirement.

Considerable research has addressed IoT device authentication through mechanisms tailored to the severe power and memory constraints of embedded hardware. Physical Unclonable Functions (PUFs), hash-based challenge–response protocols, and symmetric-key constructions have emerged as dominant lightweight approaches owing to their low computational overhead. DAuth [1] follows this direction: it combines PUFs with a hash-based accumulator, eliminates long-term key storage, distributes authentication across multiple in-network nodes, and supports multihop verification at minimal overhead.

Despite these properties, a fundamental privacy gap persists. During each authentication session, the device's real identity ID_D is transmitted in plaintext over the public channel. A passive adversary can capture these transmissions to track the device across sessions, correlate repeated authentication attempts, and infer network behaviour, which

directly violates modern privacy requirements.

Identity privacy has been established as a fundamental property in IoT authentication, not merely an optional feature. Survey literature confirms that anonymity, untraceability, and forward secrecy are essential, particularly when devices operate in open or adversarial environments [2]. Existing countermeasures such as session-refreshed pseudonyms and hash-chain-based temporary identities partially address this exposure but often introduce new challenges such as desynchronisation, excessive storage overhead, slow identity lookup, or increased computation. The sub-framework taxonomy of [2], covering SF3-UA through SF9-FS, illustrates how difficult it is to simultaneously satisfy anonymity, unlinkability, and resilience to message loss.

This work investigates how anonymity can be integrated into DAuth without compromising its lightweight properties. All authentication messages of DAuth traverse an open channel, and transmitting ID_D in plaintext exposes the device to tracking, session linkage, and targeted impersonation. Drawing on prior anonymous-authentication literature, this work identifies a low-overhead pseudonym mechanism that integrates cleanly into DAuth. The objective is identity anonymity, session unlinkability, and non-traceability, achieved without relaxing the lightweight and distributed properties of DAuth.

A second gap in DAuth concerns desynchronisation resilience. Because DAuth retains only a single session nonce at the authentication server, any loss of the key-exchange response packet causes the device and server states to diverge, leading to a permanent authentication failure that can only be resolved through costly full re-enrolment.

1.1 Contributions

This work makes the following key contributions.

1. An anonymity-preserving scheme is proposed that introduces per-session pseudonym rotation to preserve device anonymity and a dual-state mechanism to recover from

desynchronisation without re-enrolment.

2. The security of the proposed scheme is validated through both informal analysis and formal verification using ProVerif.

3. The efficiency of the proposed scheme is evaluated using theoretical analysis, COOJA/ Contiki-NG simulation, and hardware experiments on Raspberry Pi 4B, covering communication overhead, energy consumption, and CPU utilisation.

1.2 Organisation of The Report

The rest of this report is organised as follows. Chapter 2 surveys the related literature. Chapter 3 describes the prerequisite concepts. Chapter 4 overviews DAuth [1]. Chapter 5 presents the proposed scheme in full detail. Chapter 6 contains the security analysis. Chapter 7 presents the performance evaluation. Chapter 8 concludes the report.

Chapter 2

Literature Survey

IoT authentication research has increasingly favoured lightweight and distributed designs, driven by the prohibitive load that centralised systems impose on a single gateway and the single point of failure they introduce. Given the severe power and memory constraints of IoT devices, the research community has sought architectures in which multiple network nodes share the authentication workload.

Light-SAE [3] exemplifies this trend. It enables already-authenticated nodes to assist newcomers through a simple challenge-and-response join protocol, using TinyJAMBU for lightweight encryption and a modified Diffie–Hellman exchange to establish the session key in multihop networks, thereby reducing the load on the gateway. However, Light-SAE transmits node identities in plaintext and provides no protection for anonymity or session unlinkability, rendering it unsuitable for privacy-sensitive deployments.

SAKMS [4] targets industrial IoT systems operating under the 6TiSCH schedule, employing ECC-based mutual authentication with ECQV certificates, formalised key management, and strong forward secrecy, with correctness confirmed by ProVerif. Its reliance on multiple elliptic-curve operations, however, imposes overhead that exceeds what severely constrained IoT nodes can sustain. SAKMS also embeds fixed device identifiers within messages, leaving devices susceptible to tracking across sessions. These limitations illus-

trate that cryptographic strength alone does not imply privacy preservation or lightweight execution.

The **DAuth PUF-based authentication scheme** [1] addresses many of these weaknesses. It eliminates long-term secret storage by deriving device secrets from PUFs, uses a hash-based accumulator for membership verification, and decouples the authentication server from the parent node so that both parent and non-parent nodes can serve as authenticators, distributing load in multihop networks. Its computational primitives (hash, XOR, and two AES calls) keep the per-device footprint minimal. Its primary limitation is that the device identity (ID_D) is transmitted directly in every authentication exchange; since all messages traverse a public channel, a passive observer can capture the identity, track the device over time, and mount targeted attacks. Identity concealment is therefore a necessary extension to this protocol.

Several anonymous authentication schemes have been proposed to address such privacy concerns. **He et al.** [5] achieve lightweight anonymous authentication using hash-and-XOR primitives for CoAP/OSCORE but rely on a centralised server, forfeiting the decoupled-distribution advantage. **LAAKA** [6] employs ECC to protect device identity during mutual authentication; ECC operations, however, incur higher power consumption than may be tolerable on severely constrained nodes.

A comprehensive survey on lightweight anonymous authentication [2] catalogues approaches including temporary pseudonyms, PUF-assisted anonymous login, hidden membership proofs via accumulators, and unlinkable tokens, while documenting common pitfalls such as desynchronisation, pseudonym exhaustion, and inadequate replay protection. Their taxonomy deconstructs existing protocols into nine sub-frameworks (SF1-UA through SF9-FS) and two general frameworks (GF1, GF2), finding that only GF2 simultaneously achieves user anonymity, forward secrecy, and desynchronisation resistance.

Building on this taxonomy, the present work adopts **GF2** as its target architectural pattern and realises a *modified* instantiation of it tailored to resource-constrained multihop

IoT nodes. Concretely, the proposed scheme combines the user-anonymity sub-frameworks (SF3-UA, SF5-UA, SF6-UA), which use rotating pseudonyms instead of a static identity, with the forward-secrecy sub-frameworks (SF7-FS, SF8-FS, SF9-FS), which derive each session key from independently refreshed material. Unlike the survey's reference constructions that rely on heavy cryptographic primitives, the proposed scheme uses lightweight PUF-based secret derivation and hash-XOR masking, and meets GF2's desynchronisation-resistance requirement through a dual-state recovery mechanism at both the authentication server and the gateway. The result is a GF2-compliant design with the same low overhead as DAuth [1].

Zhou et al. [7] propose a security-enhanced lightweight and anonymity-preserving user authentication scheme for IoT-based healthcare, providing hash-based anonymous authentication with nonce masking to protect sensitive metadata over open wireless channels.

Other works use **blockchain** [8] and **certificateless cryptography** [9] to achieve decentralisation and identity hiding, but they incur prohibitive storage and consensus overhead. **Li et al.** [10] and **Zheng et al.** [11] combine PUF with session key exchange but rely on ECC operations too expensive for highly constrained nodes.

Despite this body of work, no existing scheme simultaneously satisfies all of the following requirements: (i) decoupled authentication, (ii) PUF-based secrecy without long-term key storage, (iii) low-overhead hash-XOR computation, and (iv) device anonymity with session unlinkability and desynchronisation recovery. This gap motivates the present work, which builds an anonymous and desynchronisation-resilient extension of DAuth [1] while preserving its lightweight and distributed properties.

To show the differences between all these methods clearly, Table 2.1 lists the use of decoupling, PUF, accumulator-based authentication, authentication secret update, anonymity, desynchronisation recovery, and the computation and communication overhead for all the schemes studied.

Table 2.1: Features Comparison of Authentication Schemes.

Scheme	Dec.	PUF	Acc.	ASU	Anon.	Desync	C.OH	M.OH
RAAF-MEC [12]	×	✓	×	✓	×	×	H	L
SAKMS [4]	✓	×	×	×	×	×	H	L
Li et al. [10]	✓	✓	×	✓	×	×	H	L
Zheng et al. [11]	✓	✓	×	✓	×	×	M	L
DAuth [1]	✓	✓	✓	✓	×	×	L	L
Proposed	✓	✓	✓	✓	✓	✓	**L**	**L**

Dec.: Decoupled design. *Acc.*: Hash-based accumulator. *ASU*: Auth. secret update.

Anon.: Device anonymity via pseudonym. *Desync*: Desync. recovery.

C.OH: Computation: L (Hash/XOR/$\leq$2 AES), M (>2 AES), H (ECC/Pairings).

M.OH: Message exchanges: L (1–3), M (4–5), H (>5).

Chapter 3

Prerequisites for Objective

3.1 Physically Unclonable Function

A Physically Unclonable Function (PUF) is a one-way physical function that generates a unique response R for a given challenge C from the challenge space $\mathcal{C}$. PUFs exploit manufacturing variations in integrated circuits that are practically impossible to replicate. Various types of PUFs have been proposed, including SRAM PUFs, Arbiter PUFs, and Ring Oscillator (RO) PUFs. In this work, a Ring Oscillator PUF is used due to its higher execution speed and lower overhead.

A function is considered a valid and secure PUF if it satisfies the following properties:

- **Evaluatable:** A legitimate entity can easily compute a response $R \in \mathcal{R}$ for any given challenge $C \in \mathcal{C}$.
- **Unpredictability:** An adversary cannot predict the response R for any challenge C without physical access to the PUF.
- **Uniqueness:** For two different PUF instances, PUF_1 and PUF_2, the probability that $\mathsf{PUF}_1(C) = \mathsf{PUF}_2(C)$ for the same challenge C is negligible.
- **Reliability:** For a fixed challenge C, the PUF response $\mathsf{PUF}(C) = R$ remains stable

over time and environmental variations.

- **One-wayness:** Given a response R, it is computationally infeasible to determine the corresponding challenge C or invert the PUF.

These properties make PUFs suitable as hardware-bound secrets for key generation, device binding, and authentication in resource-constrained IoT devices.

3.2 Hash-based One-Way Accumulator

A one-way hash accumulator, proposed as an efficient alternative to digital signatures in authentication protocols, provides a compact, non-invertible representation of a set of elements. In such an accumulator, items are combined using a quasi-commutative operator, so the final accumulated value does not depend on the order of insertion.

Formally, a function

$$f : A \times B \to A$$

is said to be quasi-commutative if, for all $a \in A$ and for all $b, c \in B$,

$$f(f(a, b), c) = f(f(a, c), b). \tag{3.1}$$

Nyberg introduced a fast accumulated hashing method suitable for lightweight systems. Let N denote the maximum number of accumulable items, and let $h : \{0,1\}^* \to \{0,1\}^\ell$ be a cryptographically secure hash function. Given a set of items $Y = \{y_1, y_2, \ldots, y_m\}$, $m \leq N$, each item is first hashed as $Y_i = h(y_i)$, $i = 1, 2, \ldots, m$. The accumulated hash is then computed as:

$$T_{\mathsf{Acc}} = H(s, Y) = s \bigodot_{i=1}^{m} h(y_i), \tag{3.2}$$

where s is an initialisation seed and $\bigodot$ represents the bitwise AND operator. Since the AND operator is commutative and the hash function is one-way, the accumulator inherits

quasi-commutative and one-way properties.

Membership Verification: To verify whether an item y_i belongs to the accumulated set, compute $Y_i = h(y_i)$, and check whether $Y_i \bigodot T_{\mathsf{Acc}} = T_{\mathsf{Acc}}$.

The security of the accumulator depends on the difficulty of forging a value Y_i that passes verification. For a security parameter t, the required length r of the accumulated hash is given by $r = N \times e \times t$, where e is Euler's (Napier's) constant.

3.3 Core Concept of Anonymity Preserving

In IoT security, anonymity is defined by two key properties: **Untraceability** (concealing the sender's real identity) and **Unlinkability** (preventing the correlation of multiple sessions to a single device). While simply encrypting a static ID protects the credential itself, it does not prevent traffic analysis; if the encrypted value remains constant, an adversary can track the device's activity patterns.

To achieve robust anonymity, modern schemes utilise **Dynamic Pseudonyms**. Instead of a static identifier, the device generates a session-specific alias (PID) using a shared secret or random nonce, e.g., $PID_{i+1} = H(ID \| Nonce_i)$. To an external observer, each authentication request appears as a fresh, random string of bits, completely uncorrelated with previous sessions.

A critical challenge when using dynamic pseudonyms is **desynchronisation**. If the message carrying the new nonce is lost in transit, the device retains the old nonce while the server has already advanced to the new one. Their pseudonym computations then diverge, and all subsequent authentications fail. To handle this gracefully, a dual-state storage approach retains both the current and the immediately previous state at the server, allowing recovery without re-enrolment.

Chapter 4

DAuth: The Base Authentication Scheme

4.1 Overview of DAuth

DAuth [1] is designed for multihop IoT networks where newly joined nodes must authenticate themselves securely before communicating with the gateway. Instead of relying on a centralised authority, the gateway distributes the authentication workload by selecting a set of already-authenticated IoT nodes to operate as Authentication Servers (AS). Let this set be represented as $\mathcal{AS}' = \{AS_1, AS_2, \ldots, AS_n\}$, where the gateway shares a long-term secret key $K_{GW\text{-}AS}$ with each AS_i.

When a new device D joins the network, the gateway (through the parent node) provides D with the address and identity ID_{AS} of the responsible authenticator. Importantly, the AS may lie at a single-hop or multi-hop distance, and does not need to be the parent node. This *decoupling* reduces the computational burden on both gateway and parent nodes, improving scalability in dense IoT environments.

The operation of the scheme is divided into two major phases: the **Node Enrollment Phase** and the **Node Authentication and Key Exchange Phase**.

4.1.1 Node Enrollment Phase

In the enrollment phase, node D communicates with its AS over a secure channel. During enrollment, D generates its unique secret y_D, hashes it to obtain $Y_D = H(y_D)$, and registers this value with the AS. The AS incorporates Y_D into its hash-based accumulator using $T_{\mathsf{Acc}} = T_{\mathsf{Acc}}$ & Y_D. To avoid storing large CRP databases, the AS stores only a lightweight entry consisting of the challenge $c_{AS\text{-}D}$ and a masked PUF response $\Phi_D = R_{AS\text{-}D} \oplus R_D$, while D stores its own challenge c_D. This significantly reduces memory usage and prevents direct leakage of raw PUF responses. Figure 4.1 illustrates this phase.

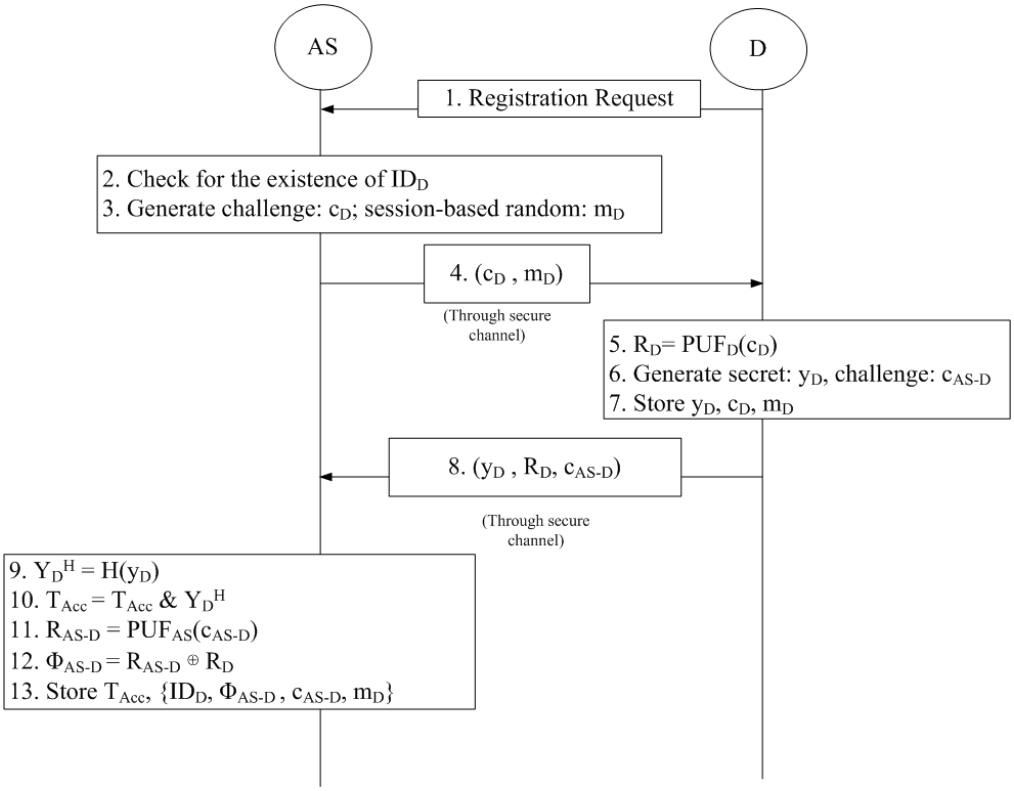


Fig. 4.1: Node Enrollment Phase of DAuth (adapted from [1]).

4.1.2 Node Authentication and Key Exchange Phase

In the **Node Authentication Phase**, node D authenticates itself to the AS over an insecure public channel. D regenerates its PUF response $R_D = \mathsf{PUF}_D(c_D)$ and computes a mask $H(R_D \| m_D \| ID_D \| ts_1)$, where m_D is the current session-based random obtained during

enrollment. The hashed witness $Y_D^H = H(y_D)$ is XORed with this mask and transmitted to the AS along with the real device identity ID_D and timestamp ts_1.

Upon receiving the message, AS reconstructs R_D using $\Phi_D \oplus R_{AS\text{-}D}$, regenerates the mask, unmasks $Y_D^{H'}$, and verifies authenticity by checking T_{Acc} & $Y_D^{H'} = T_{\mathsf{Acc}}$. If equality holds, the AS confirms that D was previously registered. Figure 4.2 shows the authentication phase.

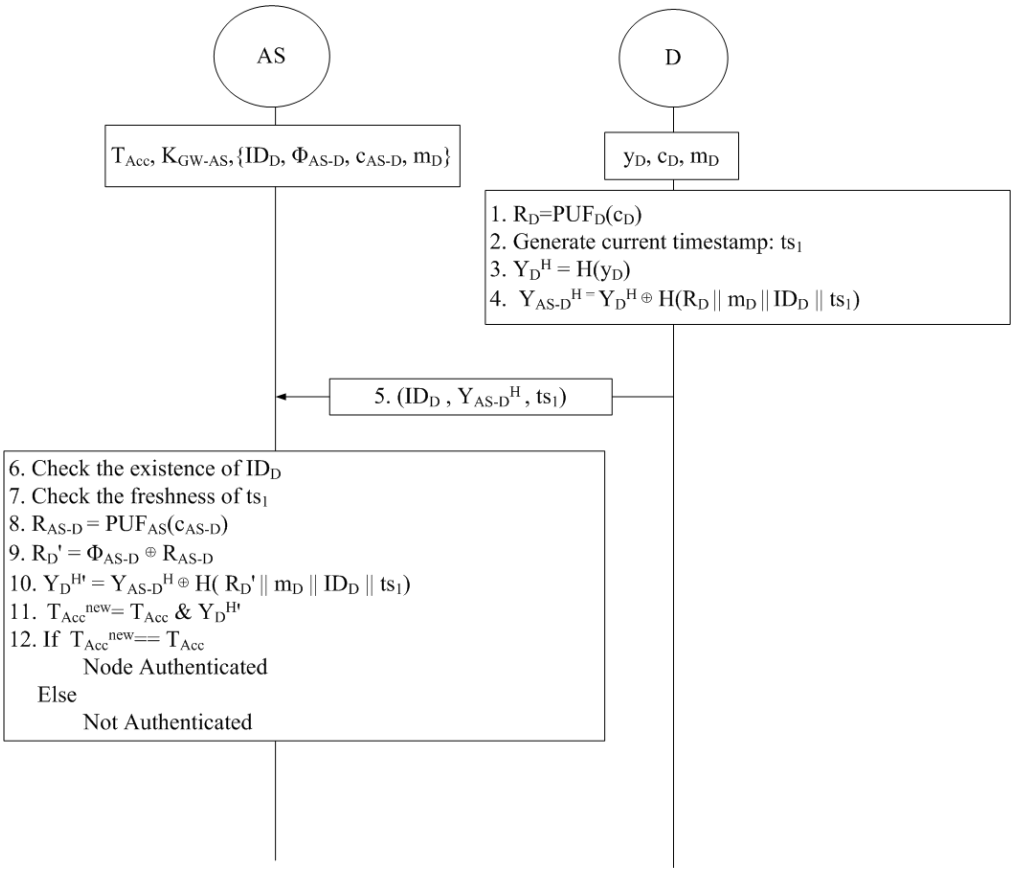


Fig. 4.2: Node Authentication Phase of DAuth (adapted from [1]).

The AS then generates a new session-based random $m_{\text{new}} = H(n_1)$, where n_1 is freshly chosen, replacing m_D for the next session and providing forward secrecy. The session key for secure communication between the gateway and the node is computed as $K_{GW\text{-}D} = H(R_D \| m_{\text{new}})$. AS constructs an authentication token containing ID_D, ID_{AS}, timestamp ts_{auth}, and the session key $K_{GW\text{-}D}$, encrypted under $K_{GW\text{-}AS}$, and sends it to the gateway. Figure 4.3 illustrates this key exchange.

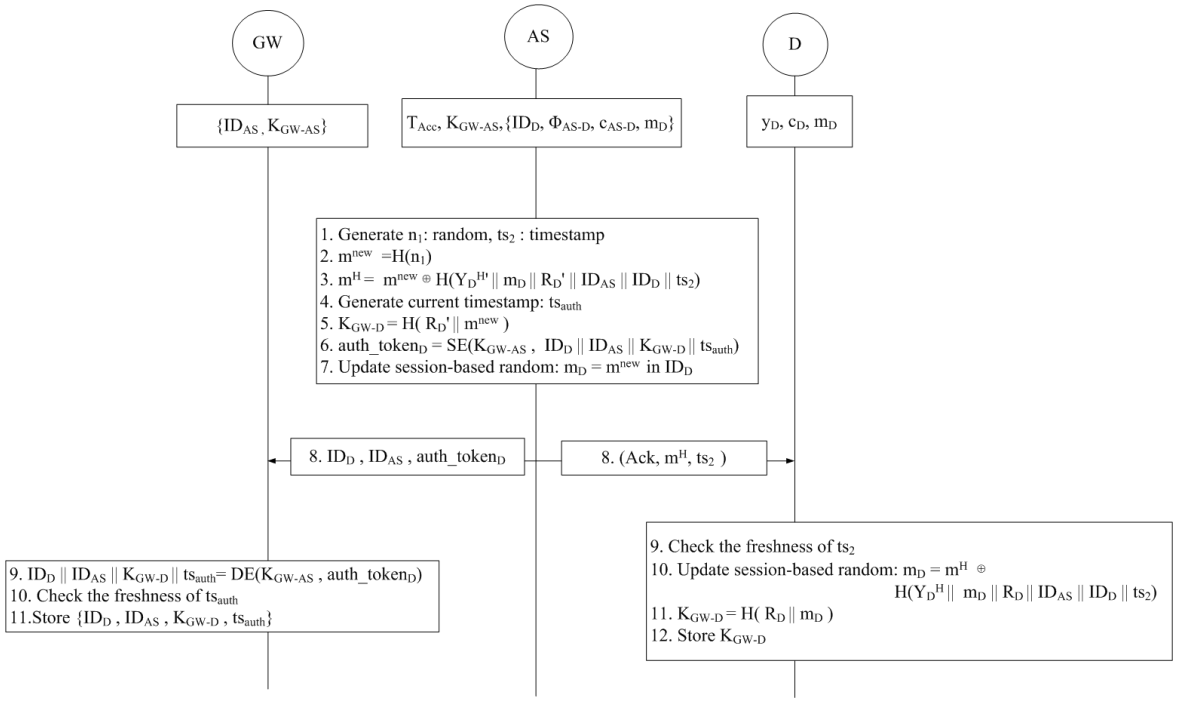


Fig. 4.3: Node Key Exchange Phase of DAuth (adapted from [1]).

4.1.3 Privacy Gaps in DAuth

Although DAuth achieves decoupled and distributed authentication, PUF-based security, lightweight computation, low storage overhead, and dynamic key updates, it leaves two critical gaps unaddressed.

Gap 1: Identity Exposure The real device identity ID_D is transmitted in plaintext on the open channel in every authentication session. In the open wireless environments typical of IoT deployments, a passive attacker monitoring the channel can readily capture these identity transmissions. Even when all other packet fields carry opaque values, the presence of a plaintext ID_D is sufficient for an adversary to link multiple authentication attempts to the same device. Accumulated over time, such captures enable long-term device tracking, session linkage, topology inference, and targeted denial-of-service against specific nodes. In a multihop network, the exposure is amplified, since each intermediate hop introduces an additional vantage point from which the identity can be observed.

Gap 2: Desynchronisation DAuth stores only a single session nonce m_D at the AS.

If the key-exchange response carrying the new m_{new} to device D is lost, D retains the old m_D while the AS has already advanced to m_{new}. On the next authentication attempt, D constructs its authentication message using the stale m_D, but the AS uses m_{new} to verify; the masks no longer match, authentication fails, and the device must undergo full re-enrolment.

These two deficiencies motivate the proposed scheme, which hides device identities from the open channel, prevents session linkage, and adds desynchronisation resilience while preserving the lightweight and distributed character of DAuth.

Chapter 5

Proposed Authentication Scheme

5.1 System Model and Assumptions

The proposed scheme is designed to help nodes establish secure multihop communications with the gateway. To establish a secure communication, nodes should be authenticated by a trusted authentication server. A set of existing IoT nodes inside the network is entrusted with the responsibility of authentication by the gateway in a distributed manner. The gateway also shares a secret key with each of those responsible IoT nodes/authentication servers. The authentication server can be a parent or a non-parent node, which may be *decoupled*, preventing newly joined child nodes from overwhelming their parent with biased authentication load.

IoT nodes are assumed to be embedded with a PUF and are capable of performing various cryptographic operations, such as encryption/decryption, XOR, hashing, bitwise AND, and concatenation.

Nodes D and AS are assumed to be the newly joined node and its authentication server, respectively. GW represents the gateway symbolically. AS may be located at a single-hop or multi-hop distance from D. It is assumed that GW has already selected the authentication server set and has established a shared secret key with each member of the set. Let $\mathcal{AS}' = \{AS_1, AS_2, \ldots, AS_n\}$ be the set of authentication servers selected

by GW, such that $AS \in \mathcal{AS}'$. GW has a shared secret key $K_{GW\text{-}AS}$ with AS. It is also assumed that D, whose distinct identifier is ID_D, possesses the address of its authentication server AS along with its unique identity ID_{AS}. Table 5.1 elaborates symbols and functions incorporated in the following scheme.

Table 5.1: Notation Summary.

Symbol	Description
D, AS	Newly joined device and its authentication server
GW	Gateway / root node
$H(\cdot)$	Collision-resistant hash function (SHA-256)
$\mathsf{SE}(\cdot)$, $\mathsf{SD}(\cdot)$	Symmetric encryption/decryption (AES-128)
$\|$	Concatenation operation
$\oplus$	Bitwise XOR operation
&	Bitwise AND operation
c_D, $c_{AS\text{-}D}$	PUF challenges for D (issued by AS) and AS (by D)
m_D	Session-based random issued by AS during enrolment
m_{curr}, m_{old}	Current and previous session-based randoms at D and AS
y_D, R_D	Unique secret and PUF response of D
Y_D^H	Hashed unique secret: $H(y_D)$
T_{Acc}	Accumulated authentication value at AS
$\Phi_{AS\text{-}D}$	PUF-response binding: $\mathsf{PUF}_{AS}(c_{AS\text{-}D}) \oplus R_D$
PID_{curr}, PID_{old}	Current and previous pseudonyms of D
$K_{GW\text{-}D}$	Session key between GW and D
$K_{GW\text{-}AS}$	Long-term shared key between GW and AS
ts_1, ts_2, ts_{auth}	Timestamps for freshness verification

There are mainly three phases: node enrolment, node authentication, and node session key exchange. The following sections elaborate on these phases.

5.2 Node Enrolment Phase

The node enrolment phase is the first phase undergone by a newly joined node D with its authentication server AS. Before entering this phase, it is assumed that D has the required information regarding AS: the address and unique identity (ID_{AS}) of AS. AS can lie at a single-hop or multi-hop distance from D. This phase is assumed to be executed in a secure environment by establishing an end-to-end encrypted channel between D and AS.

Every newly joined node executes this phase once until it disconnects from the network. Figure 5.1 outlines the methodology of this phase.

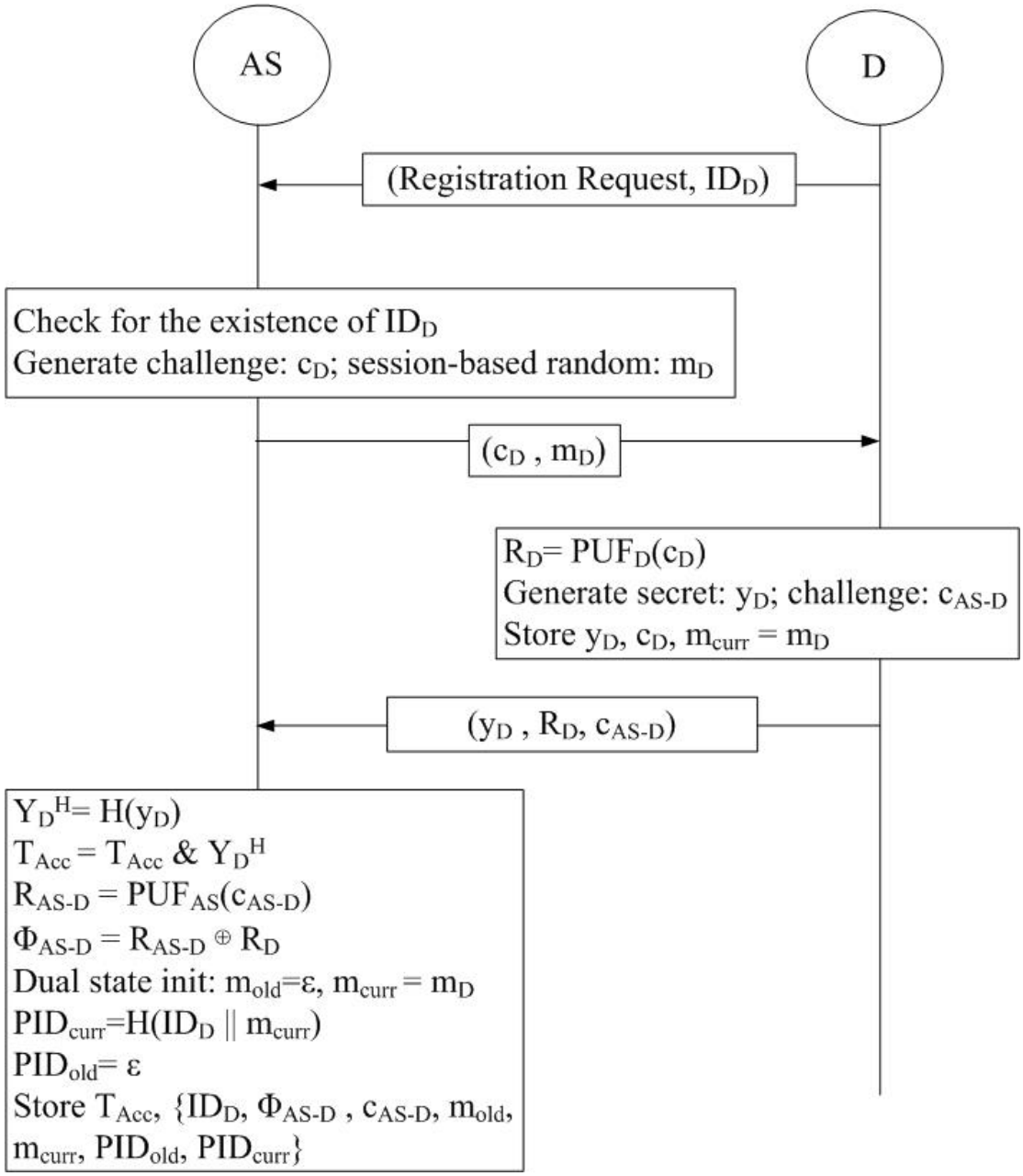


Fig. 5.1: Proposed Scheme: Node Enrolment Phase with pseudonym initialisation.

D initiates the phase by sending a registration request along with its unique identity ID_D to AS over the secure channel. As a response, AS generates a challenge c_D from the predefined challenge set $\mathbb{C}_D$ and a session-based random m_D; these are then forwarded

securely to D. With c_D as input to its embedded PUF_D, D obtains a unique response $R_D = \mathsf{PUF}_D(c_D)$. After this, D generates its own unique secret value y_D, also a random number; the security of the entire scheme critically depends on maintaining the confidentiality of y_D. D also picks a reverse challenge $c_{AS\text{-}D}$ from the predefined challenge set $\mathbb{C}_{AS\text{-}D}$ and sends $(y_D, R_D, c_{AS\text{-}D})$ back to AS through the secure channel. D then stores $\{y_D, c_D, m_D\}$. Unlike [1], which stores m_D as a plain session random, the proposed scheme retains it as the initial *current* session random m_{curr}, seeding the pseudonym mechanism.

AS inputs the secret y_D to the hash function $H(\cdot)$, generating tag $Y_D^H = H(y_D)$, and updates the accumulator as $T_{\mathsf{Acc}} = T_{\mathsf{Acc}} \,\&\, Y_D^H$. Further, AS generates $R_{AS\text{-}D} = \mathsf{PUF}_{AS}(c_{AS\text{-}D})$ and calculates $\Phi_{AS\text{-}D} = R_{AS\text{-}D} \oplus R_D$.

Pseudonym Initialisation (new in proposed scheme): The proposed scheme extends this base enrolment with pseudonym and dual-state initialisation to eliminate ID_D from all future open-channel messages. Since ID_D is transmitted only over this secure enrolment channel and must never appear on the open channel again, D will henceforth be identified by the rotating pseudonym $PID = H(ID_D \| m_{\text{curr}})$ in all future authentication messages. Because H is one-way and m_{curr} is a private session random, successive pseudonyms are computationally unlinkable by a channel observer. AS pre-computes $PID_{\text{curr}} = H(ID_D \| m_{\text{curr}})$ and sets $PID_{\text{old}} = \perp$, $m_{\text{old}} = \perp$, forming the initial *dual state*. AS therefore stores $\{T_{\mathsf{Acc}}, ID_D, \Phi_{AS\text{-}D}, c_{AS\text{-}D}, m_{\text{curr}}, m_{\text{old}}, PID_{\text{curr}}, PID_{\text{old}}\}$.

5.3 Node Authentication Phase

The node authentication phase is the second phase of the proposed scheme. Its main purpose is to authenticate D's unique secret y_D with AS. Since this phase is executed over an open channel, y_D must be communicated in secure form, preventing it from being intercepted by adversaries. This security is established through PUF-based challenge-response, producing a secret key R_D, and the session-based random m_{curr}, incorporating unpredictability. Both R_D and m_{curr} are known only to D and AS. Figure 5.2 illustrates the

phase.

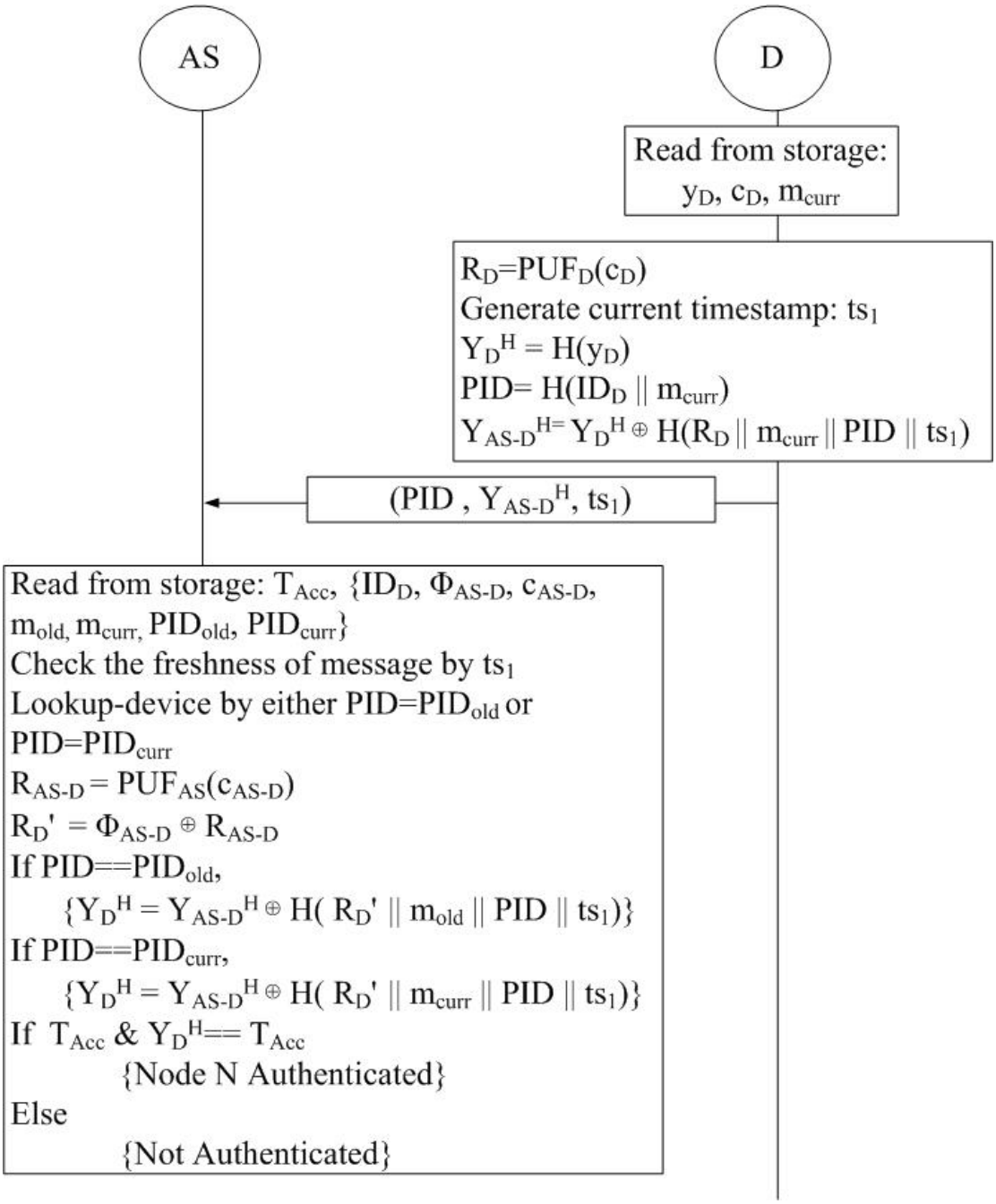


Fig. 5.2: Proposed Scheme: Node Authentication Phase with dual-state pseudonym lookup.

The phase is initiated by D. D regenerates $R_D = \mathsf{PUF}_D(c_D)$ from the stored challenge c_D obtained during enrolment. D then computes its current pseudonym $PID = H(ID_D \| m_{\text{curr}})$, which is used in place of the real identity ID_D in the outgoing message. D

produces the hashed secret $Y_D^H = H(y_D)$ in XORed form as:

$$Y_{AS\text{-}D}^H = H(y_D) \oplus H(R_D \| m_{\text{curr}} \| PID \| ts_1). \tag{5.1}$$

The XOR mask is the hash of R_D, the session-based random m_{curr}, the pseudonym PID, and the current timestamp ts_1. The timestamp is used to provide replay protection to the XORed value $Y_{AS\text{-}D}^H$. The complete message $(PID, Y_{AS\text{-}D}^H, ts_1)$ is then sent to AS.

After receiving the message, AS checks for the existence of an entry matching the received PID. In a departure from [1], AS performs a **dual-state lookup**: it checks whether the received PID matches PID_{curr} or PID_{old} in its records. A match on PID_{old} indicates that D missed the Key Exchange Phase delivery in the previous session and still holds the prior m_{curr}; in this case, AS uses the stored m_{old} for the subsequent verification, recovering from the desynchronisation without requiring re-enrolment. AS verifies the freshness of ts_1, reconstructs R'_D as $\Phi_{AS\text{-}D} \oplus \mathsf{PUF}_{AS}(c_{AS\text{-}D})$, regenerates the same XOR mask, unmasks $Y_D^{H'}$, and then performs the bitwise AND check:

$$T_{\mathsf{Acc}}^{\mathsf{new}} == T_{\mathsf{Acc}} \;\&\; Y_D^{H'} \stackrel{?}{=} T_{\mathsf{Acc}}. \tag{5.2}$$

If $T_{\mathsf{Acc}}^{\mathsf{new}} == T_{\mathsf{Acc}}$, it proves that the unique secret of D is already a legitimate member of AS. With the successful operation, the authentication of D is completed by AS, and AS proceeds to the Key Exchange Phase.

5.4 Node Session Key Exchange Phase

This phase is executed by AS to inform GW about the authentication of the newly joined node D at the end of the authentication phase. AS uses an $\mathsf{auth\_token}_D$ to deliver the session key $K_{GW\text{-}D}$ to GW, facilitating session key generation between GW and D. The proposed scheme extends [1] by additionally deriving a fresh pseudonym $PID_{\text{new}} = H(ID_D \| m_{\text{new}})$

at the end of each session and carrying it within $\mathsf{auth\_token}_D$, so that GW can update its pseudonym mapping for D and successive sessions remain unlinkable. Figure 5.3 shows the phase.

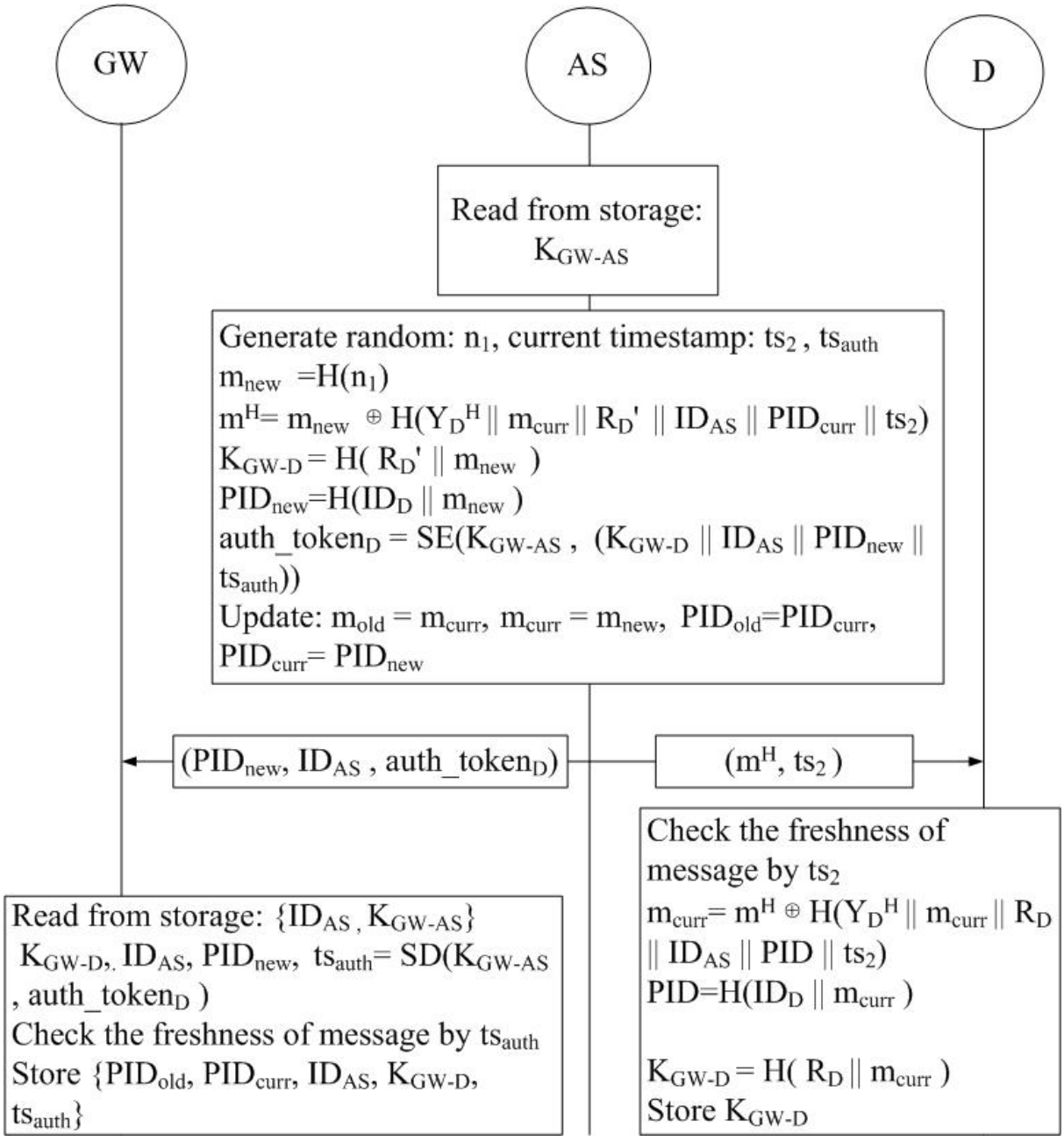


Fig. 5.3: Proposed Scheme: Node Session Key Exchange Phase with pseudonym rotation.

$K_{GW\text{-}D}$ is calculated by AS using the hashed output of R'_D and m_{new}:

$$K_{GW\text{-}D} = H(R'_D \| m_{\text{new}}). \tag{5.3}$$

m_{new} is computed by hashing a fresh random number n_1 generated by AS as $m_{\text{new}} = H(n_1)$. Further, AS derives $PID_{\text{new}} = H(ID_D \| m_{\text{new}})$. The session key $K_{GW\text{-}D}$, the identities ID_{AS} and ID_D, the next pseudonym PID_{new}, and the authentication completion timestamp ts_{auth} are encrypted under the shared key $K_{GW\text{-}AS}$ to form $\mathsf{auth\_token}_D$. AS simultaneously sends (m_H, ts_2) to D and $(PID_{\text{new}}, ID_{AS}, \mathsf{auth\_token}_D)$ to GW, where the masked session seed is:

$$m_H = m_{\text{new}} \oplus H(Y_D^{H'} \| m_{\text{curr}} \| R'_D \| ID_{AS} \| PID_{\text{curr}} \| ts_2). \tag{5.4}$$

It then advances its records: $m_{\text{curr}} \leftarrow m_{\text{new}}$, $PID_{\text{curr}} \leftarrow PID_{\text{new}}$, with m_{old} and PID_{old} retaining the superseded values.

Obtaining (m_H, ts_2), D first verifies the freshness of ts_2, then recovers m_{new} by re-generating the hash mask using its locally held Y_D^H, R_D, m_{curr}, ID_{AS}, PID_{curr}, and the received ts_2, and XORing with m_H. D establishes $K_{GW\text{-}D} = H(R_D \| m_{\text{new}})$ and rotates its pseudonym to $PID_{\text{curr}} = H(ID_D \| m_{\text{new}})$, thus preparing itself for secure communication with GW.

GW decrypts the token with $K_{GW\text{-}AS}$, verifies ts_{auth}, and extracts $K_{GW\text{-}D}$ and PID_{new}. GW saves PID_{curr} as the previous pseudonym, sets PID_{new} as the current one, and stores the session key against both.

5.5 Desynchronisation Recovery

During the Key Exchange Phase, AS simultaneously dispatches two packets: one to GW carrying $(PID_{\text{new}}, ID_{AS}, \mathsf{auth\_token}_D)$, and one to D carrying (m_H, ts_2). Loss of either packet leads to a different desynchronisation scenario, each handled without re-enrolment.

If the $AS \rightarrow D$ packet (m_H, ts_2) is lost, D retains its stale m_{curr}, so the pseudonym it computes in the next session equals PID_{old} from AS's perspective. AS recognises this via the dual-state lookup and resumes from the consistent prior state, restoring synchronisation without any re-enrolment overhead.

If instead the $AS \rightarrow GW$ packet $(PID_{\text{new}}, ID_{AS}, \mathsf{auth\_token}_D)$ is lost, D has already advanced its local state: it holds m_{new} and $PID_{\text{curr}} = PID_{\text{new}}$ after successfully receiving (m_H, ts_2). GW, however, holds no valid session key for D. When D subsequently attempts data communication and GW cannot verify it, D re-initiates from the Authentication Phase using its current PID_{curr} (i.e., PID_{new}). AS matches this against PID_{curr} in its dual-state record and follows the normal authentication path, generating a fresh $\mathsf{auth\_token}_D$ and forwarding it to GW, which then correctly establishes the session key. No re-enrolment is required in this scenario either.

Algorithm 1 shows how the scheme adapts the Authentication and Key Exchange Phases to recover from the $AS \rightarrow D$ loss scenario without re-enrolment.

The dual-state mechanism tolerates a single consecutive Key Exchange Phase loss. Two or more consecutive losses exhaust both slots and require re-enrolment, which is a deliberate trade-off between storage overhead and recovery depth.

Algorithm 1 Desynchronisation Recovery

Precondition: D holds stale $m_{\text{curr}} = m_{\text{old}}$; AS holds $(m_{\text{curr}},\, m_{\text{old}},\, PID_{\text{curr}},\, PID_{\text{old}})$

Authentication Phase

$PID \leftarrow H(ID_D \| m_{\text{curr}})$ ▷ stale $m_{\text{curr}} \Rightarrow PID = PID_{\text{old}}$
$D \rightarrow AS$: $(PID,\, Y^{H}_{AS\text{-}D},\, ts_1)$
if $PID = PID_{\text{curr}}$ **then**
    AS verifies using m_{curr} ▷ normal path
else if $PID = PID_{\text{old}}$ **then**
    AS verifies using m_{old} ▷ desync detected
else
    AS rejects D
end if

Key Exchange Phase (desync path)

$n_1 \leftarrow \mathsf{rand}()$
$m_{\text{new}} \leftarrow H(n_1)$
$m_H \leftarrow m_{\text{new}} \oplus H\big(Y_D^{H'} \| m_{\text{old}} \| R'_D \| ID_{AS} \| PID_{\text{old}} \| ts_2\big)$
$AS \rightarrow D$: $(m_H,\, ts_2)$
$m_{\text{new}} \leftarrow m_H \oplus H\big(Y_D^{H} \| m_{\text{curr}} \| R_D \| ID_{AS} \| PID \| ts_2\big)$ ▷ D uses own $m_{\text{curr}} = m_{\text{old}}$
$m_{\text{old}} \leftarrow m_{\text{curr}}$; $\quad m_{\text{curr}} \leftarrow m_{\text{new}}$; $\quad PID \leftarrow H(ID_D \| m_{\text{curr}})$

Result: D and AS re-synchronised; PID rotated

Chapter 6

Security Analysis

6.1 Formal Security Analysis

The proposed scheme is translated into ProVerif [13] queries under the Dolev–Yao adversary model [14], where the adversary has full control of the public channel and can intercept, replay, modify, and inject messages. All 10 queries are satisfied. Figure 6.1 shows the verification output.

```
------------------------------------------------------------
Verification summary:

Query inj-event(DeviceEnrolled(id_D_2,id_AS_3)) ==> inj-event(DeviceEnrollmentStarts(id_D_2,id_AS_3)) is true.

Query inj-event(DeviceAuthenticated(id_D_2,id_AS_3)) ==> inj-event(DeviceAuthenticationStarts(id_D_2,id_AS_3)) is true.

Query inj-event(AuthenticationServerEnds(id_AS_3,id_D_2)) ==> inj-event(AuthenticationServerStarts(id_AS_3,id_D_2)) is
true.

Query inj-event(AuthenticationEndsFull(id_D_2,id_AS_3,R_D_2,m_D_2)) ==> inj-event(AuthenticationStartsFull(id_D_2,id_AS
_3,R_D_2,m_D_2)) is true.

Query event(DeviceKeyExDone(id_D_2,id_AS_3)) ==> event(DeviceKeyExStarts(id_D_2,id_AS_3)) is true.

Query not attacker(SecretK_GW_D_Device[]) is true.

Query not attacker(SecretK_GW_D_GW[]) is true.

Query not attacker(SecretM_New[]) is true.

Query not attacker(SecretID_D[]) is true.

Weak secret SecretK_GW_D_Device is true.
```

Fig. 6.1: ProVerif verification output: all 10 queries satisfied.

6.1.1 Correspondence Queries (Q1–Q5)

Five correspondence queries verify that protocol steps occur in the correct order and that no adversary can skip or forge a step.

Q1 (Enrolment correspondence): A device can only be enrolled at the authentication server after it has explicitly initiated the enrolment process. This ensures a device can only be enrolled with its own participation.

Q2 (Authentication, device side): The device receives the server's acknowledgement and the fresh timestamp ts_2 only after it first sent a valid authentication request. This prevents replay of a stale acknowledgement.

Q3 (Authentication, server side): The authentication server completes its processing only after receiving a valid request from the device. This rules out an adversary forging a server-side completion event.

Q4 (Two-round binding): Q4 binds the Authentication and Key Exchange phases: the PUF response R_D and the session seed m_{curr} committed during authentication are proven to be the same values used in the key exchange. This rules out credential substitution between phases.

Q5 (Key exchange correspondence): The device recovers the new session seed m_{new} only after it legitimately sent the key exchange request.

6.1.2 Secrecy and Forward Secrecy (Q6–Q9)

Q6–Q7 (Session key secrecy): Two queries verify the secrecy of the session key $K_{GW\text{-}D} = H(R_D \| m_{\text{new}})$ from the device and gateway perspectives. Even though the adversary observes all traffic on the public channel, it cannot reconstruct $K_{GW\text{-}D}$ because the PUF response R_D is never transmitted and m_{new} is always delivered XOR-masked.

Q8 (Forward secrecy of m_{new}): This query confirms that the rotated session seed m_{new} remains secret. Even if an adversary recovers a current session key, it gains nothing about future sessions because each m_{new} is derived from an independently generated fresh

nonce.

Q9 (ID_D anonymity): After enrolment, ID_D is never sent over the open channel again. Every subsequent message carries the pseudonym $PID = H(ID_D \| m_{\text{curr}})$. ProVerif confirms that no attacker, even one who intercepts every message on the network, can recover ID_D from the observed traffic.

Anonymity is meaningful only if successive pseudonyms are unlinkable. Since Q8 proves m_{new} is secret, consecutive pseudonyms $H(ID_D \| m_{\text{curr}})$ and $H(ID_D \| m_{\text{new}})$ are computationally indistinguishable to any observer.

6.1.3 Offline Guessing Resistance (Q10)

Q10 (Weak secrecy): This weak-secrecy query confirms that the session key $K_{GW\text{-}D}$ cannot be guessed from the public transcript even by an adversary with unlimited offline dictionary power. This protects against brute-force and dictionary attacks that try to recover the key from captured packets.

6.2 Informal Analysis

6.2.1 Replay Attack Resistance

The Enrolment Phase is executed over a secure channel and is immune to replay. The Authentication and Key Exchange Phases embed timestamps (ts_1, ts_2, ts_{auth}) and the session-based random m_{curr} into the hash masks. A replayed message with a stale timestamp or stale m_{curr} will fail the freshness check at AS, D, or GW respectively.

6.2.2 MITM Attack Resistance

The Enrolment Phase is conducted over an encrypted channel, which prevents MITM attacks. In the Authentication and Key Exchange Phases, the PUF response R_D (device-specific, unreproducible) and session random m_{curr} (private to D and AS) are embedded

in every XOR mask. Without R_D and the current m_{curr}, no adversary can forge a valid masked tag $Y^H_{AS\text{-}D}$ or unmask m_H.

6.2.3 Device Anonymity

ID_D is transmitted only once, over the secure enrolment channel, and is never exposed on the open network again. All open-channel messages use $PID = H(ID_D \| m_{\text{curr}})$, which is refreshed to $H(ID_D \| m_{\text{new}})$ after every key exchange. Because m_{new} is derived from a freshly generated nonce n_1, successive pseudonyms are computationally unlinkable.

6.2.4 Desynchronisation Recovery

Packet loss can occur at three distinct points in the Authentication and Key Exchange Phases, each handled without requiring re-enrolment.

If the acknowledgement and fresh timestamp ts_2 sent by AS to D at the end of the authentication phase are lost in transit, D does not advance its internal state. Since AS has not yet performed any state update at this stage, both entities remain consistent. Upon timeout, D retransmits the authentication request with the same PID and ts_1, and the exchange proceeds normally.

The primary desynchronisation scenario arises when the Key Exchange Phase response (m_H, ts_2) is lost. In this case, AS has already advanced its state to m_{new} and rotated the pseudonym to PID_{new}, while D retains the previous m_{curr}. On the next authentication attempt, D constructs its pseudonym from the stale m_{curr}, producing PID_{old}. AS recognises this via the dual-state lookup and resumes from the consistent prior state.

If the token forwarded from AS to GW is lost, D re-initiates from the Authentication Phase; the dual-state storage at AS ensures that this re-authentication succeeds and a fresh token is forwarded to GW.

6.2.5 Forward Secrecy

$K_{GW\text{-}D} = H(R_D \| m_{\text{new}})$. Compromise of a session key does not reveal previous session keys because m_{new} is independently generated from a fresh nonce and R_D is PUF-derived.

Chapter 7

Performance Analysis

7.1 Theoretical Performance Analysis

7.1.1 Computational Cost

Computational cost directly determines how efficiently an authentication scheme can be deployed on resource-constrained hardware. The Python cryptographic library `pycrypto` is used in a Windows environment to measure the computational cost of core cryptographic functions, such as SHA-256 (T_{hash}), AES Encryption/Decryption (T_{aes}), and random generation (T_{rand}). The PUF operation cost (T_{puf}) is taken from prior benchmarks on the same platform [11]. Benchmark averages over 100 iterations are: $T_{\mathsf{rand}} = 0.0018\,\text{ms}$, $T_{\mathsf{puf}} = 0.0522\,\text{ms}$, $T_{\mathsf{hash}} = 0.0353\,\text{ms}$, $T_{\mathsf{aes}} = 0.0867\,\text{ms}$.

Enrolment Phase

To execute the node enrolment phase:

- AS computes two random generations (T_{rand}), one PUF operation (T_{puf}), and two hash operations (T_{hash}) for pseudonym initialisation.
- D executes two random generations (T_{rand}) and one PUF operation (T_{puf}).

Hence, the total computational cost per node enrolment amounts to $2T_{\mathsf{puf}} + 2T_{\mathsf{hash}} + 4T_{\mathsf{rand}} = \mathbf{0.18}$ ms. The proposed scheme incurs a marginal overhead of 0.03 ms over DAuth [1] (0.15 ms), directly attributable to the additional hash operation for pseudonym initialisation, a security feature absent from DAuth. Table 7.1 compares the enrolment computational cost.

Table 7.1: Computational Cost Comparison: Enrolment Phase.

Scheme	Operations	Cost (ms)
LAAKA [6]	$3T_{\mathsf{hash}}$	0.11
Zhou [7]	$T_{\mathsf{puf}} + 2T_{\mathsf{hash}} + 3T_{\mathsf{rand}}$	0.13
Zheng et al. [11]	$2T_{\mathsf{puf}} + 8T_{\mathsf{hash}} + 4T_{\mathsf{aes}} + 5T_{\mathsf{rand}}$	0.69
He et al. [5]	$4T_{\mathsf{hash}} + T_{\mathsf{aes}} + T_{\mathsf{rand}}$	0.23
DAuth [1]	$2T_{\mathsf{puf}} + 1T_{\mathsf{hash}} + 4T_{\mathsf{rand}}$	0.15
Proposed	$2T_{\mathsf{puf}} + 2T_{\mathsf{hash}} + 4T_{\mathsf{rand}}$	**0.18**

Benchmark (avg. 100 iterations): T_{rand}=0.0018 ms, T_{puf}=0.0522 ms, T_{hash}=0.0353 ms, T_{aes}=0.0867 ms.

Authentication and Key Exchange Phase

In the proposed scheme, D and AS both execute one PUF operation (T_{puf}).

- D additionally performs six hash operations (T_{hash}): pseudonym computation, unique secret hash, and XOR mask in the Authentication Phase, followed by mask recovery, session key derivation, and pseudonym rotation in the Key Exchange Phase.
- AS performs five hash operations (T_{hash}), along with one AES encryption (T_{aes}) for the gateway token and one random generation (T_{rand}) for m_{new}.
- GW runs one AES decryption (T_{aes}) to recover the session key and updated pseudonym from the forwarded token.

This leads to a total cost of $2T_{\mathsf{puf}} + 11T_{\mathsf{hash}} + 2T_{\mathsf{aes}} + T_{\mathsf{rand}} = \mathbf{0.67}$ ms. The proposed scheme incurs a modest overhead of 0.11 ms over DAuth [1] (0.56 ms), attributable to three additional hash operations for pseudonym computation, dual-state lookup, and pseudonym

rotation, which is the price paid for anonymity and desynchronisation resilience. Table 7.2 compares the authentication and key exchange computational cost.

Table 7.2: Computational Cost Comparison: Authentication & Key Exchange Phase.

Scheme	Operations	Cost (ms)
LAAKA [6]	$16T_{\mathsf{hash}}$	0.56
Zhou [7]	$15T_{\mathsf{hash}}$	0.53
Zheng et al. [11]	$2T_{\mathsf{puf}} + 6T_{\mathsf{hash}} + 4T_{\mathsf{aes}} + 5T_{\mathsf{rand}}$	0.67
He et al. [5]	$11T_{\mathsf{hash}} + 5T_{\mathsf{aes}} + 2T_{\mathsf{ecc}}$	1.74
DAuth [1]	$2T_{\mathsf{puf}} + 8T_{\mathsf{hash}} + 2T_{\mathsf{aes}} + T_{\mathsf{rand}}$	0.56
Proposed	$2T_{\mathsf{puf}} + 11T_{\mathsf{hash}} + 2T_{\mathsf{aes}} + T_{\mathsf{rand}}$	**0.67**

Benchmark (avg. 100 iterations): T_{rand}=0.0018 ms, T_{puf}=0.0522 ms, T_{hash}=0.0353 ms, T_{aes}=0.0867 ms; T_{ecc}=0.461 ms (scaled from He et al. [5] hardware ratios).

7.1.2 Communication Cost

Communication cost is a core feature for analysing the performance of an authentication scheme, based on the number of message exchanges and their sizes (in bits). It is assumed that identities and timestamps are 32 bits, along with nonces, hashes, challenges, responses, and ciphertexts as 256 bits, ensuring a fair comparison.

Enrolment Phase

During the node enrolment phase, D enrols itself with AS, executing three message exchanges: (1) ID_D (32 b) from D to AS; (2) c_D (256 b) and m_D (256 b) from AS to D; (3) y_D (256 b), R_D (256 b), and $c_{AS\text{-}D}$ (256 b) from D to AS. Total enrolment communication cost: **1312 bits**. Table 7.3 shows the comparison.

Authentication and Key Exchange Phase

The authentication and key exchange phase is executed using three message communications. Message 1 ($D \rightarrow AS$): PID (256 b) + $Y^H_{AS\text{-}D}$ (256 b) + ts_1 (32 b). Message 2

Table 7.3: Communication Cost Comparison: Enrolment Phase.

Scheme	# Messages	Cost (bits)
LAAKA [6]	3	2592
Zhou [7]	5	1344
Zheng et al. [11]	3	1600
He et al. [5]	2	1280
DAuth [1]	3	1312
Proposed	3	**1312**

IDs/timestamps = 32 b; hashes/nonces/challenges/ciphertexts = 256 b.

($AS \rightarrow D$): m_H (256 b) + ts_2 (32 b). Message 3 ($AS \rightarrow GW$): PID_{new} (256 b) + ID_{AS} (32 b) + $\mathsf{auth\_token}_D$ (256 b). Total: **1376 bits**. Table 7.4 shows the comparison.

Table 7.4: Communication Cost Comparison: Authentication & Key Exchange Phase.

Scheme	# Messages	Cost (bits)
LAAKA [6]	3	2400
Zhou [7]	4	2464
Zheng et al. [11]	3	1600
He et al. [5]	3	1120
DAuth [1]	3	928
Proposed	3	**1376**

IDs/timestamps = 32 b; hashes/nonces/challenges/ciphertexts = 256 b.

DAuth [1] requires 928 bits; the 448-bit increase in the proposed scheme directly reflects the widening of the device identifier from a 32-bit ID_D to a 256-bit pseudonym PID, which is the direct cost of adding device anonymity to all open-channel communications. The proposed scheme matches DAuth exactly in the enrolment phase (1312 bits), demonstrating that pseudonym initialisation adds no communication overhead over the DAuth at registration.

7.2 Simulation-Based Performance Analysis

A simulation-based performance evaluation has been conducted using the COOJA simulator within the Contiki-NG framework. Three complementary studies are reported.

1. **Overall per-device comparison:** Proposed scheme vs. DAuth [1], LAAKA [6], and Zhou [7] in a fixed 100-node network (1 GW, 79 AS nodes with 2 active, 20 newly joined devices).

2. **AS pool-size study:** Active AS nodes varied from 2 to 10.

3. **Network scalability study:** Network size N varied across $\{30, 50, 80, 100, 120\}$ nodes, with 20 % of nodes as newly joined devices.

All results are averaged over ten independent random seeds. Table 7.5 lists the common simulation parameters.

Table 7.5: Simulation Parameters.

Parameter	Value
Operating System	Contiki-NG
Mote Type	Cooja Mote
Network Size	100 Nodes
Propagation Model	UDGM (Distance Loss)
Application Layer	CoAP
Network Layer	RPL Lite
MAC Layer	CSMA / IEEE 802.15.4
Simulation Time	75 Seconds
No. of Simulations	10 per configuration
Schemes compared	Proposed, DAuth [1], LAAKA [6], Zhou [7]
Active AS (study ii)	2, 5, 10
Network size (study iii)	$N \in \{30, 50, 80, 100, 120\}$

7.2.1 Overall Per-Device Cost

For 20 newly joined nodes in a fixed 100-node network with two active authentication servers, the proposed scheme demonstrated a consistent advantage over LAAKA and Zhou. As shown in Figure 7.1(a), the proposed scheme achieves a reduction of **42.8 %** and **32.4 %** in per-device energy consumption over LAAKA and Zhou et al., respectively. Similarly, a reduction of **42.8 %** and **32.4 %** is demonstrated in per-device CPU time (Figure 7.1(b)). DAuth [1] sits just below the proposed scheme in both metrics, as it requires

no pseudonym operations; the proposed scheme pays a small overhead (approximately 8 %) for the pseudonym-rotation and dual-state mechanisms that deliver anonymity and desynchronisation resilience, both of which are absent from DAuth. Due to the incorporation of fewer message exchanges and computationally lightweight hash-XOR operations, the proposed scheme has minimised radio activity and computational load compared to the message-intensive and cryptographically heavier LAAKA and Zhou et al. schemes.

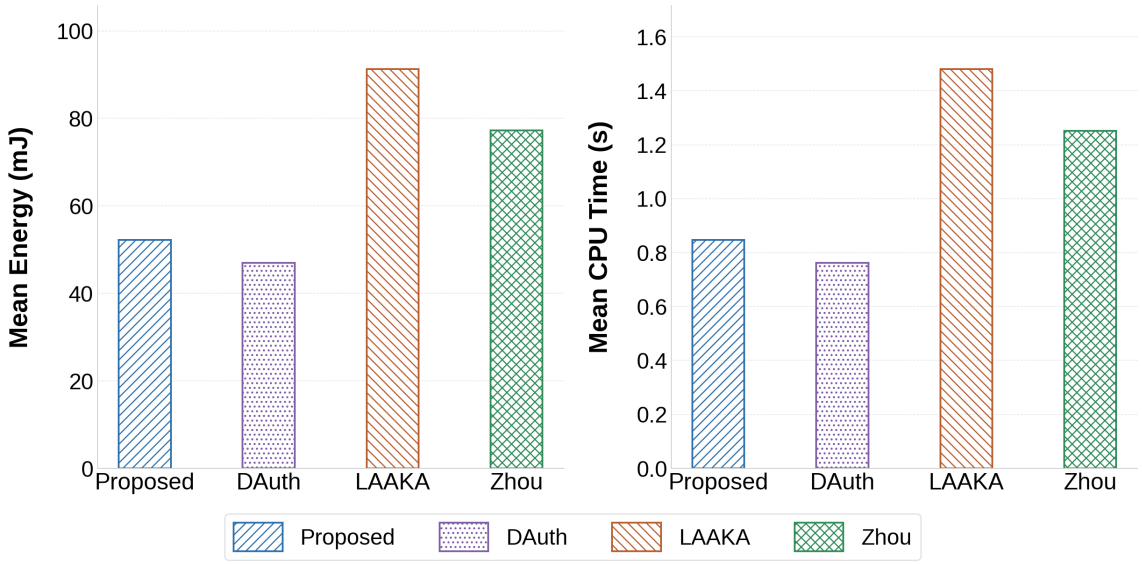


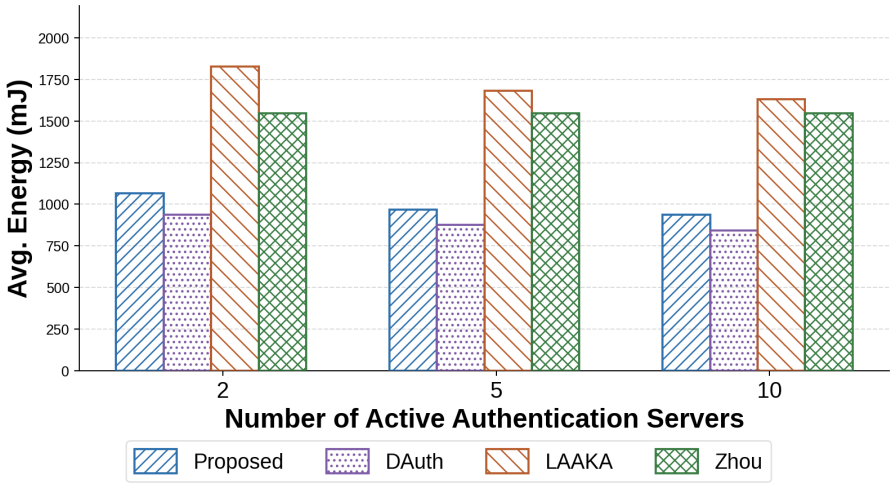
Fig. 7.1: Per-device mean total cost, all phases: (a) energy and (b) CPU time, 100-node network, 10 seeds.

7.2.2 Authentication Server Pool Size

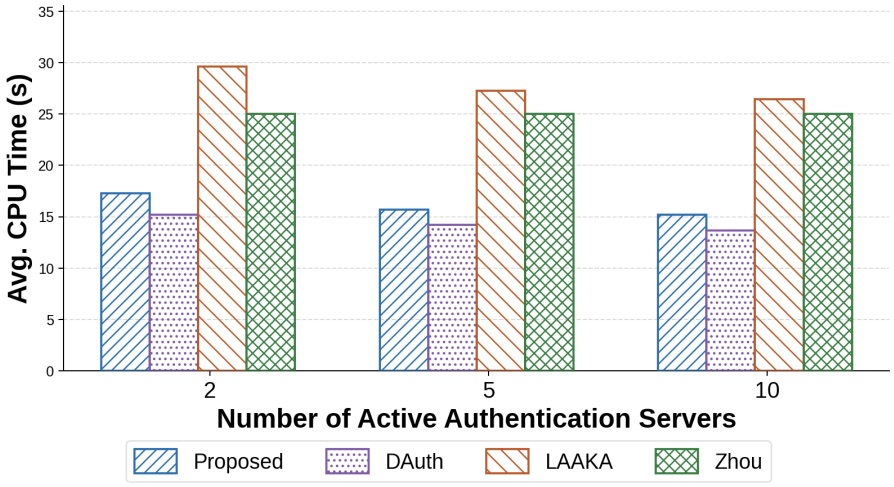
In the decoupled design, the gateway selects any capable node as the authentication server, so the size of the AS pool directly influences load distribution and overall network performance. The performance of the proposed scheme, DAuth, LAAKA, and Zhou is evaluated by varying the number of active AS nodes from 2 to 10.

Considering the total energy consumption (Figure 7.2(a)), a reduction of approximately **42 %** is demonstrated by the proposed scheme over LAAKA at every AS count. Both the proposed scheme and DAuth improve as the pool grows from 2 to 10 servers, since additional authenticators absorb the join load of newly arrived devices. As shown in Figure 7.2(b), a

reduction of approximately 42 % in total CPU time is demonstrated over LAAKA across all AS counts. Zhou et al. shows essentially flat cost across all AS pool sizes. DAuth sits consistently just below the proposed scheme, reflecting the absence of pseudonym-rotation overhead.



(a) Total energy vs. number of active AS nodes.



(b) Total CPU time vs. number of active AS nodes.

Fig. 7.2: Authentication cost vs. AS pool size (20 devices, 5 seeds).

7.2.3 Network Scalability

The performance of authentication schemes is also analysed by considering their scalability with respect to total network size N, varied from 30 to 120 nodes, where 20 % of nodes

in each network are newly joined devices. Considering the total energy consumption (Figure 7.3(a)), at $N = 120$ the proposed scheme demonstrates a reduction of **42.7 %** over LAAKA and **37.6 %** over Zhou et al. Similarly, as shown in Figure 7.3(b), a reduction of 42.7 % and 37.6 % is demonstrated with respect to total CPU time over LAAKA and Zhou et al., respectively, at $N = 120$. LAAKA and Zhou et al. exhibit steeper energy and CPU growth with increasing N because their heavier per-device cryptographic operations accumulate faster as the device count grows. DAuth and the proposed scheme both scale favourably, maintaining the lowest total cost across all network sizes; the proposed scheme incurs only a modest overhead above DAuth attributable to the additional pseudonym-rotation mechanism.

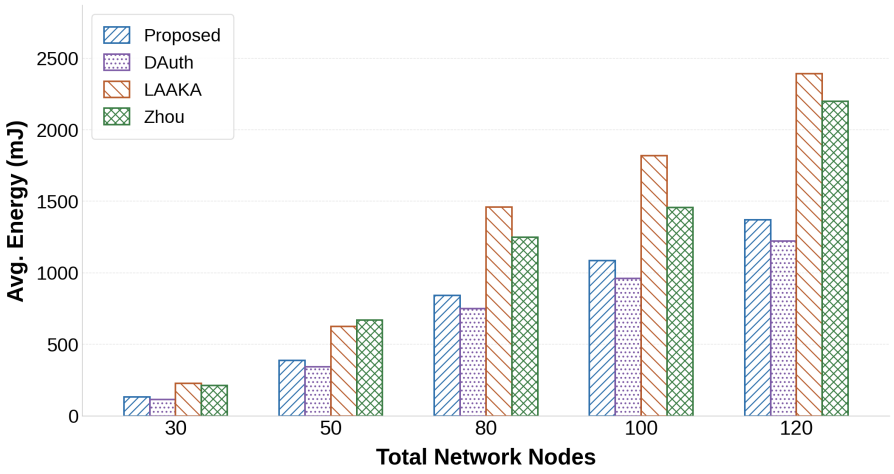
7.2.4 Desynchronisation Recovery

During the Key Exchange Phase, the AS simultaneously dispatches two packets: one to the gateway carrying $(PID_{\text{new}}, ID_{AS}, \mathsf{auth\_token}_D)$, and one to the device carrying (m_H, ts_2). Desynchronisation arises when the packet destined for the device is lost in transit.

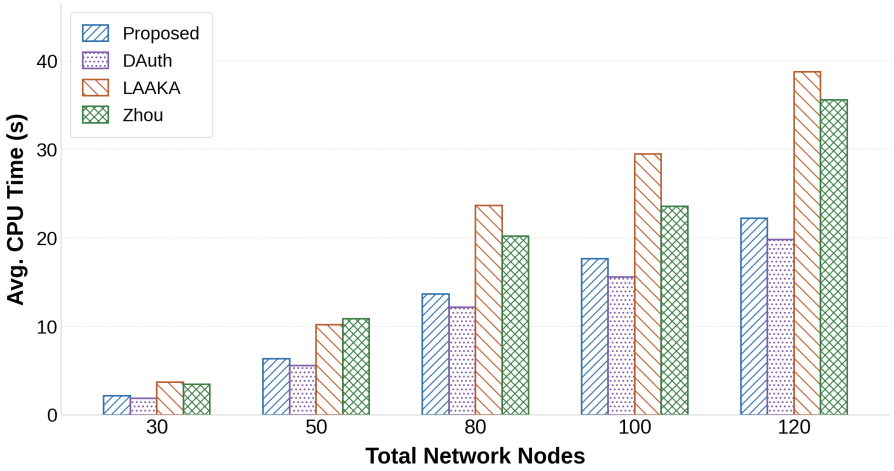
DAuth [1] stores only a single nonce state at the AS. When the device re-authenticates using the old m_{curr}, the AS has no means to recognise it and rejects the request. The device must therefore undergo a full re-enrolment before a new authentication and key exchange can succeed. The proposed scheme eliminates this penalty through its dual-state mechanism.

Since the dual-state desynchronisation recovery mechanism is a direct extension of DAuth's single-state protocol, only DAuth serves as the meaningful baseline for this comparison.

Figure 7.4 presents a per-device energy and CPU time comparison of a single authentication round under two conditions: normal operation (before any packet loss) and recovery (the round immediately following a desynchronisation event), evaluated over a 100-node multi-hop RPL network, 20 devices, and 5 seeds. As shown in Figure 7.4(a), DAuth [1] in-



(a) Total energy vs. network size N.



(b) Total CPU time vs. network size N.

Fig. 7.3: Network scalability: total energy and CPU time vs. N.

curs 134.8 mJ per recovery round, a 118.3 % increase over its normal round cost of 61.8 mJ, reflecting the overhead of a failed authentication attempt followed by full re-enrolment. The proposed scheme's recovery round costs 87.2 mJ, statistically indistinguishable from its normal round (98.6 mJ), confirming that the dual-state mechanism incurs zero additional overhead during recovery. Compared directly, the proposed scheme reduces per-device recovery energy by **47.6 mJ (35.3 %)** over DAuth [1]. The CPU time results in Figure 7.4(b) exhibit the same pattern: DAuth [1] incurs 2.185 s per recovery round against 1.413 s for the proposed scheme, yielding a **35.3 % reduction** in execution time.

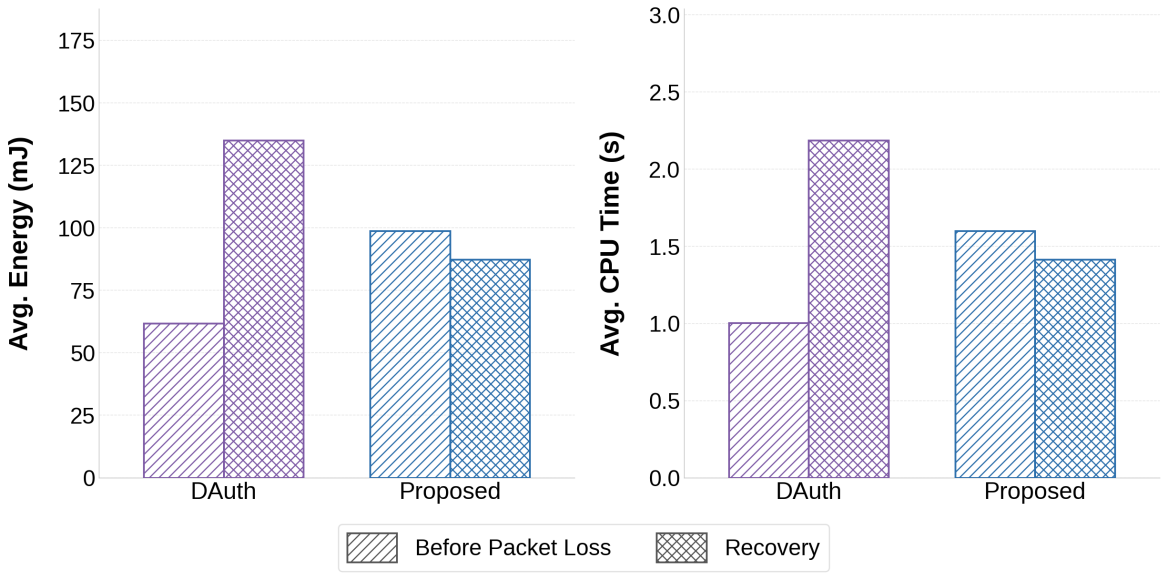


Fig. 7.4: Per-device desynchronisation recovery cost: (a) energy and (b) CPU time for a normal round vs. a recovery round, averaged over 20 devices and 5 seeds in a 100-node RPL network.

7.3 Hardware-Based Performance Analysis

Hardware-based evaluation validates the feasibility of the proposed scheme by measuring its performance on physical devices under real operating conditions.

All four schemes (the proposed scheme, DAuth [1], LAAKA [6], and Zhou et al. [7]) are implemented on a **Raspberry Pi 4B** platform, equipped with a quad-core 64-bit ARM Cortex-A72 processor operating at 1.5 GHz. Identical hardware is used across all four schemes to ensure a fair comparison of energy consumption and CPU time. The implementation considers two Raspberry Pi 4B platforms as IoT nodes: one performing the role of the newly joined device, and the other functioning as the authentication server. A Windows laptop is leveraged as the gateway. Each of the three entities maintains direct TCP communication with the others. Energy consumption is estimated using the active power profile of the Raspberry Pi 4B (3.8 W) multiplied by the measured wall-clock time per authentication round. The physical testbed is shown in Figure 7.5.

As illustrated in Figure 7.6(a), DAuth achieves the lowest per-round energy of **0.152 J**,

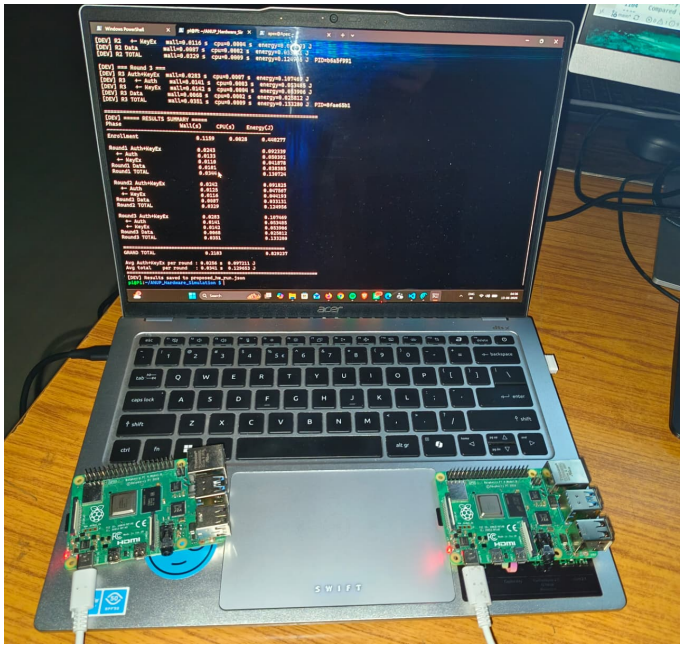


Fig. 7.5: Hardware testbed: two Raspberry Pi 4B boards (left: authentication server; right: device) connected to a Windows laptop (gateway).

as it performs no pseudonym operations. The proposed scheme consumes **0.286 J** per round, which is approximately 88 % more than DAuth due to the additional SHA-256 evaluations for pseudonym rotation and dual-state verification. Despite this overhead, the proposed scheme remains **14.2 %** and **52.5 %** more energy-efficient than Zhou et al. (0.333 J) and LAAKA (0.602 J), respectively, confirming that the anonymity guarantee is achieved at a cost well within the range of existing non-anonymous schemes.

The CPU time results in Figure 7.6(b) follow the same ordering: DAuth (0.040 s), Proposed (0.075 s), Zhou et al. (0.088 s), and LAAKA (0.159 s). The proposed scheme reduces CPU time by **14.8 %** and **52.8 %** relative to Zhou et al. and LAAKA, respectively, while incurring an 87.5 % overhead compared to DAuth, consistent with the energy results. These reductions are consistent with the lightweight XOR-hash operations and decoupled AS design of the proposed scheme, which translate directly to measurable gains in physical energy and execution time on real hardware.

The inconsistency between hardware and simulation-based results arises due to the disparity in scale, where a 100-node network is assumed in simulation whereas the hardware

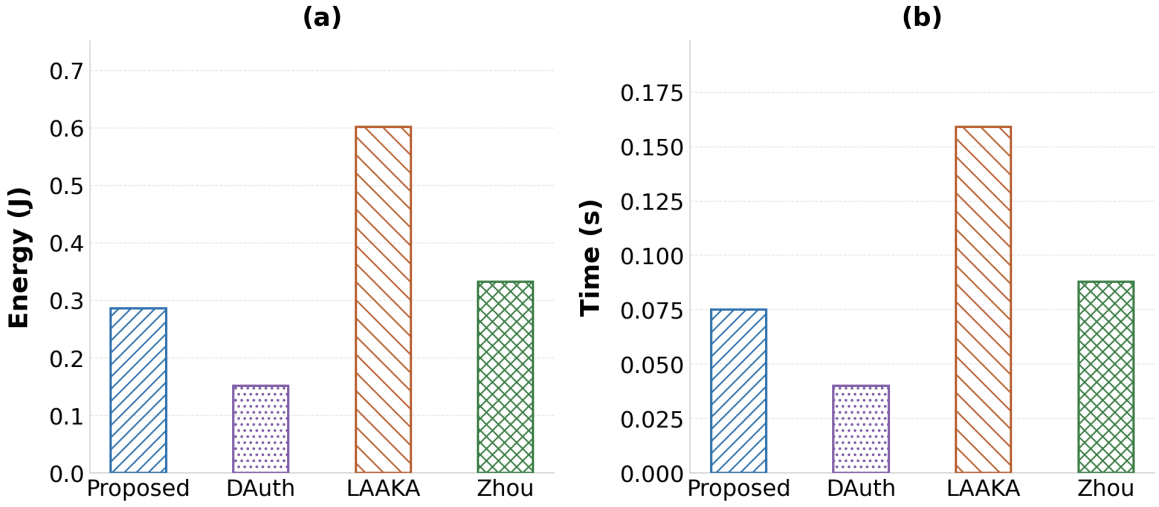


Fig. 7.6: Hardware-based performance: (a) energy consumption and (b) CPU time per authentication round on Raspberry Pi 4B (Proposed vs. DAuth vs. LAAKA vs. Zhou et al.).

setup involves only two Raspberry Pi platforms.

Chapter 8

Conclusion

This report presented a lightweight anonymous authentication scheme for multihop IoT networks that extends DAuth [1] with two additions: per-session pseudonym rotation to hide ID_D from the open channel, and a dual-state mechanism to recover from desynchronisation without re-enrolment.

Formal verification in ProVerif confirms all 10 security queries under the Dolev–Yao model, covering authentication correspondence, session key secrecy, forward secrecy, anonymity, and offline guessing resistance.

COOJA/Contiki-NG simulations on 100-node networks show per-device energy reductions of **42.8 %** over LAAKA and **32.4 %** over Zhou et al. The dual-state mechanism is particularly significant in the desynchronised case: DAuth incurs a **118.3 %** energy penalty per recovery round (134.8 mJ vs. 61.8 mJ for a normal round) because a single lost packet forces full re-enrolment, whereas the proposed scheme's recovery round costs 87.2 mJ (close to its normal round of 98.6 mJ), saving **35.3 %** in both energy and CPU time over DAuth.

Hardware experiments on Raspberry Pi 4B confirm per-round energy reductions of **52.5 %** over LAAKA and **14.2 %** over Zhou et al., with an 88 % overhead above DAuth attributable solely to the additional hash operations for pseudonym rotation.

Among the four compared schemes, the proposed scheme is the only one to simultaneously provide device anonymity, desynchronisation recovery, and decoupled distributed authentication at overhead suitable for resource-constrained IoT devices.

References

[1] S. Das and M. Khatua, "Lightweight and decoupled distributed authentication for multihop IoT networks," in *Proc. IEEE COMSNETS*, 2026.

[2] J. Zhong, S. He, Z. Liu, and L. Xiong, "Lightweight anonymous authentication for IoT: A taxonomy and survey of security frameworks," *Sensors*, vol. 25, no. 17, p. 5594, 2025.

[3] P. Rosa, A. Souto, and J. Cecílio, "Light-SAE: A lightweight authentication protocol for large-scale IoT environments made with constrained devices," *IEEE Trans. Network and Service Management*, vol. 20, no. 3, pp. 2428–2441, 2023.

[4] W. Yang, C. Hou, Y. Wang, Z. Zhang, X. Wang, and Y. Cao, "SAKMS: A secure authentication and key management scheme for IETF 6TiSCH industrial wireless networks based on improved elliptic-curve cryptography," *IEEE Trans. Network Science and Engineering*, vol. 11, no. 3, pp. 3174–3188, 2024.

[5] D. He, Y. Cai, S. Zhu, Z. Zhao, S. Chan, and M. Guizani, "A lightweight authentication and key exchange protocol with anonymity for IoT," *IEEE Trans. Wireless Communications*, vol. 22, no. 11, pp. 7862–7872, 2023.

[6] H. Ali and I. Ahmed, "LAAKA: A lightweight anonymous authentication and key agreement scheme for IoT-fog environment," *Computers & Security*, vol. 140, p. 103770, 2024.

[7] X. Zhou, S. Wang, K. Wen, B. Hu, X. Tan, and Q. Xie, "Security-enhanced lightweight and anonymity-preserving user authentication scheme for IoT-based healthcare," *IEEE Internet of Things Journal*, vol. 11, no. 6, pp. 9718–9731, 2024.

[8] M. T. Al Ahmed, F. Hashim, S. J. Hashim, and A. Abdullah, "Authentication-chains: Blockchain-inspired lightweight authentication protocol for IoT networks," *Electronics*, vol. 12, no. 4, p. 867, 2023.

[9] S. Al-Riyami and K. G. Paterson, "Certificateless public key cryptography," in *Proc. ASIACRYPT*, ser. LNCS, vol. 2894. Springer, 2003, pp. 452–473.

[10] S. Li, T. Zhang, B. Yu, and K. He, "A provably secure and practical PUF-based end-to-end mutual authentication and key exchange protocol for IoT," *IEEE Sensors Journal*, vol. 21, no. 4, pp. 5487–5501, 2020.

[11] Y. Zheng, W. Liu, C. Gu, and C.-H. Chang, "PUF-based mutual authentication and key exchange protocol for peer-to-peer IoT applications," *IEEE Trans. Dependable and Secure Computing*, vol. 20, no. 4, pp. 3299–3316, 2022.

[12] O. Alruwaili, M. Tanveer, S. A. Aldossari, S. Alanazi, and A. Armghan, "RAAF-MEC: Reliable and anonymous authentication framework for IoT-enabled mobile edge computing environment," *Internet of Things*, vol. 29, p. 101459, 2025.

[13] B. Blanchet, B. Smyth, V. Cheval, and M. Sylvestre, "ProVerif 2.00: Automatic cryptographic protocol verifier, user manual and tutorial," INRIA, Tech. Rep., 2018.

[14] D. Dolev and A. Yao, "On the security of public key protocols," *IEEE Trans. Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.